

Ordinary FORS+C Accumulation in CEDRUS+C: A Fresh-Message Forgery within the NGCC Chosen-Message Query Budget

Martin Feussner
Selmer Center, University of Bergen
martin.feussner@uib.no

3 October 2026

Provenance and review status. The candidate-specific attack instantiation, exact concentration argument, implementations, experiments, resource analysis, release package, and initial manuscript were produced by OpenAI Codex using Daybreak Blue at Ultra reasoning effort during an AI-run audit of NGCC signature candidates. Separately instructed hostile-review agents reconstructed the strict repaired-verifier model, recomputed the probability bounds independently, rebuilt the native tests, and tried to disprove the result. They accepted it only with the novelty, scope, and physical-resource qualifications stated below. At the date above, no human cryptanalyst has independently verified the proof, implementations, measurements, or novelty assessment. This report is a preliminary disclosure for human review and reproduction.

Abstract

CEDRUS+C is a stateless hash-based signature whose bottom few-time component, FORS+C, reveals one secret leaf and authentication path in each of k fixed Merkle trees. Its specification already observes that repeated use of one bottom address accumulates leaf openings. We determine the concrete consequence when signatures at every bottom address are accumulated and the forger uses the public message-hash inputs accepted by the verifier. The concrete parameter analysis is limited to the two category-I sets, CEDRUS+C-160f and CEDRUS+C-160s.

The adversary requests ordinary independently randomized signatures on distinct chosen messages, buckets them by the complete h -bit bottom address, and retains one opening for each observed address-coordinate-leaf triple. At a fixed address, openings from different signatures reconstruct the same ordered tuple of FORS roots and hence the same FORS+C public key. One honest strict-valid hypertree suffix for that address therefore authenticates any covered coordinate mixture. For a never-signed message, the adversary enumerates serialized public $(R, \text{ctr}_{\text{FTS}})$ values until the verifier's hash selects a retained address and only covered leaves. The verifier can check the resulting hash condition, but cannot check whether R came from the signer's secret PRF_{msg} computation.

We give exact first and second coverage moments. Poissonizing the acquisition count makes complete-address loads independent, and a concentration bound yields a fixed public-hash budget and a success lower bound close to $1 - e^{-1}$. For CEDRUS+C-160f, the all-address optimum has signing mean $2^{70.928}$, fixed target budget $2^{66.640}$, and $2^{71.000}$ attacker public- H_{msg} evaluations including transcript parsing. For 160s the corresponding exponents are 74.760, 71.139, and 74.873. Prefix shards whose fully provisioned tables are below 2^{80} bytes give public-hash totals $2^{71.292}$ and $2^{75.701}$. Each row has success at least 0.63212 before a negligible hard-query-cap abort.

Those interaction counts do not make the attack physically executable. The sharded rows return $2^{88.494}$ and $2^{91.795}$ bits and cost the honest signer $2^{100.156}$ and $2^{108.597}$ measured reference cycles. Sharding a fully provisioned table to the audit host's 125.77 GiB raises the

public-hash totals to $2^{73.652}$ and $2^{79.811}$ and signer work to $2^{102.474}$ and $2^{112.010}$ cycles. We report every resource in its own unit. No full-parameter end-to-end attack was executed.

A scaled experiment under the strict repaired verifier produced an accepted fresh-message forgery from ordinary randomized signatures and rejected seven adversarial controls. Separate native experiments using the untouched submitted FORS cores confirm the full-parameter coordinate splice for 160f and 160s. The attack uses a strict, scheme-favourable interpretation of the full-width specification and does not use reset, repeated randomness, deterministic signing, weak or related keys, omitted WOTS checks, or the submitted source’s tree-index collapse. The generic FORS+C accumulation mechanism appears in CEDRUS+C Section 3.2, and public-hash amplification appears in Abri–Katz. Our claimed contribution is limited to the CEDRUS+C parameter and lifetime consequence, rigorous all-address concentration and sharding analysis, and complete resource accounting.

1 Introduction

CEDRUS+C is a stateless hash-based signature in the SPHINCS+ family [1, 2]. A hypertree authenticates a bottom FORS+C public key. The message digest selects a complete bottom address and one leaf in each of k Merkle trees. Revealing different leaves under one repeated address gradually exposes a larger fraction of the possible digest vectors.

This phenomenon is acknowledged in the submission. Section 3.2 defines the overall signing-query count q_{sig} , mixes over the occupancy of one FORS+C instance, and gives

$$p_{\text{Succ}}^{\text{FTS}}(q) = [1 - (1 - 2^{-a})q]^k 2^{-a'}. \quad (1)$$

That expression concerns a new random target after q uses of one instance. The specification does not turn it into an all-address attack, control the realized aggregate table, optimize signing and public target trials together, or state the resulting resource points for its parameters.

The serialized message randomizer supplies the missing public target search. Algorithm 18 reads R and a public counter from the candidate signature and recomputes H_{msg} . Verification has no secret input with which to prove that R originated from PRF_{msg} . An ordinary forger can therefore hold a fresh message fixed and enumerate public values until the digest is covered by the accumulated table. This verifier fact is standard for randomized hash-based signatures. Abri and Katz explicitly multiply a related occupancy attack by the number of public hash queries [3, Eq. (2)]; we do not claim that technique as new.

The concrete result is an ordinary chosen-message forgery under a strict, scheme-favourable interpretation of the specification. It excludes two easier defects in the submission: the implementation sets the hypertree index to zero, and the written WOTS+C verifier omits a membership check. We instead decode the full tree index and grant the scheme the strongest coherent WOTS+C checks. The adversary uses independent hedged signing randomness and never asks the signer to repeat a random value or state.

Our contributions are as follows.

1. We give an explicit same-address coordinate-splicing forgery and prove acceptance by the strict repaired verifier.
2. We derive exact first and second moments for the fraction of covered FORS vectors at one address. Exact Poisson splitting and Chebyshev concentration turn the expected coverage into a one-run success bound.
3. We optimize all-address and fixed-prefix attacks for the two category-I parameter sets, CEDRUS+C-160f and CEDRUS+C-160s, under a hard abort before 2^{80} signing requests. The concrete parameter analysis is limited to these two sets.
4. We keep signing queries, target hashes, transcript parsing, returned data, attacker tables, table operations, and honest-signer cycles in separate units. We also give rows fitting a fully provisioned table in the audit host’s RAM.

5. We validate the complete attack with a scaled strict repaired verifier and the splice relation with untouched submitted FORS code at the full 160f and 160s dimensions. We make no claim that the astronomical full-parameter acquisition was executed.
6. We delimit novelty to the candidate-specific parameter/lifetime consequence and its rigorous concrete accounting. Equation (1) and public-hash amplification are prior art.

2 Strict, scheme-favourable specification interpretation

2.1 Keys, ordinary randomized signing, and digest parsing

For node length n , Algorithms 16–18 define

$$\text{SK} = (\text{SK.seed}, \text{SK.prf}, \text{PK.seed}, \text{PK.root}), \quad \text{PK} = (\text{PK.seed}, \text{PK.root}). \quad (2)$$

The first three seeds are independent n -byte values, and PK.root is the top hypertree root. Ordinary hedged signing draws a fresh n -byte value addrnd and computes

$$R = \text{PRF}_{\text{msg}}(\text{SK.prf}, \text{addrnd}, M), \quad (3)$$

$$D = \text{H}_{\text{msg}}(R, \text{PK.seed}, \text{PK.root}, M, \text{ctr}_{\text{FTS}}). \quad (4)$$

The signer increments the public 32-bit counter until the designated a' bits of D are zero. The remaining disjoint fields select k independent a -bit FORS indices and the complete bottom address

$$A = (\text{layer} = 0, \text{idxTree}, \text{idxLeaf}), \quad |\text{idxTree}| + |\text{idxLeaf}| = h. \quad (5)$$

In the ideal- H_{msg} model used by the submission, conditioning on the disjoint forced field leaves A uniform on 2^h values and all FORS indices uniform and independent. Independent hedged signing queries therefore give independent samples. Table 1 records the two lowest parameter sets.

Table 1: CEDRUS+C category-I parameters used in this report.

set	n	h	d	(a, k, a')	WOTS len	$ \sigma $	sign cycles	layer heights
	bytes					bytes		
160f	20	66	17	(7, 30, 9)	40	19,812	513.49M	$15 \times 4, 2 \times 3$
160s	20	67	9	(12, 13, 15)	26	9,460	8.65G	$4 \times 8, 5 \times 7$

The cycle figures are independent NGCC x86 harness means, not estimates of attacker work. The specification separately reports 335,982,300 and 5,662,176,265 cycles. We retain the slower independent measurements for the physical-resource rows.

2.2 The fixed-address FORS+C relation

Put $B = 2^a$. For coordinate $i \in \{0, \dots, k-1\}$, leaf $x \in \{0, \dots, B-1\}$, and global leaf number $u = iB + x$, Algorithms 12–15 define

$$s_{A,i,x} = \text{PRF}(\text{PK.seed}, \text{SK.seed}, \text{ADRS}_{\text{FORS-PRF}}(A, u)), \quad (6)$$

$$L_{A,i,x} = F(\text{PK.seed}, \text{ADRS}_{\text{FORS-TREE}}(A, u, 0), s_{A,i,x}). \quad (7)$$

The B leaves for coordinate i form a height- a Merkle tree with root $T_{A,i}$. The address-specific FORS+C public key is

$$P_A = T_k(\text{PK.seed}, \text{ADRS}_{\text{FORS-ROOT}}(A), T_{A,0} \parallel \dots \parallel T_{A,k-1}). \quad (8)$$

A signature selecting x_i reveals s_{A,i,x_i} and its complete authentication path. The path reconstructs the fixed root $T_{A,i}$.

Algorithm 17 signs P_A with the bottom WOTS+C key at A . At every higher layer, the WOTS+C message is likewise a fixed structural root. A complete hypertree suffix from an honest signature at A therefore authenticates exactly the same value whenever a mixed FORS opening reconstructs P_A .

2.3 Strict WOTS+C and full-address interpretation

The submitted source's collapsed tree index is not used. We parse the full $h - h_0$ tree bits and the full h_0 leaf bits in Algorithm 18, so there are exactly 2^h bottom addresses.

We also grant CEDRUS+C the strongest coherent interpretation of its WOTS+C verification. Each serialized WOTS counter is installed in the root-hash address; the verifier checks both the leading-zero predicate and the fixed-sum digit-vector membership predicate before reconstructing the WOTS key. This strict check only strengthens the scheme. The accumulation attack copies one complete valid suffix without altering any WOTS chain. Moreover, all legal digit vectors have the same sum. If public chain advancement maps legal x to legal y , then $y_j \geq x_j$ for every coordinate; equal sums force $x = y$. Thus no omitted WOTS check or chain-advancement shortcut contributes to the result.

Minor naming and serialization slips in the written algorithms are resolved in the scheme's favor: the one-counter layout shown by Figure 1.14, Algorithm 18's length, published sizes, and source is used; stale PORS/FORS names denote Algorithms 12–15; and the defined bottom-to-top hypertree flow is used. None of these repairs creates the fixed value P_A .

3 Coordinate-splicing forgery

Lemma 1 (Same-address coordinate mixing). *Fix an ordinary key and complete address A . For every target vector $x = (x_0, \dots, x_{k-1})$, one valid opening of (A, i, x_i) may be selected independently for each coordinate i from different signatures. Their ordered combination reconstructs P_A in Equation (8).*

Proof. Every selected component reconstructs the fixed root $T_{A,i}$ of its own coordinate tree. Verification preserves coordinate order and compresses the same tuple $(T_{A,0}, \dots, T_{A,k-1})$ under the same root address. The result is therefore P_A , independently of which donor supplied each component. $\square$

Lemma 2 (Public target inputs). *For a fixed public key and message M^* , a forger may submit arbitrary serialized values (R^*, ctr^*) to Algorithm 18 and is accepted when the recomputed digest satisfies the public forced-bit, address, and opening checks.*

Proof. Algorithm 18 parses both values from the signature and recomputes Equation (4). It has neither `SK.prf` nor `addrnd` and therefore cannot test Equation (3). Signature validity is determined by the public verification checks. The 160-bit R field alone supplies more distinct inputs than any target budget below. $\square$

The adversary proceeds as follows.

1. Draw an acquisition count $N \leftarrow \$ \text{Pois}(2^q)$. Abort rather than issue signing request 2^{80} . Otherwise request N ordinary hedged signatures on distinct messages.
2. Recompute each public digest, bucket the signature by its exact complete address A , and retain one opening for every observed (A, i, x) . Retain one complete strict-valid hypertree suffix per used address. A fixed high-bit prefix may be selected in advance to shard the table.

3. Fix a message M^* never submitted to the signing oracle. Enumerate distinct public (R^*, ctr^*) values until the forced field is zero, the selected address is retained, and every selected (i, x_i^*) has an opening.
4. For each coordinate copy a matching opening, then attach the retained suffix for the same complete address.

Theorem 3 (Fresh-message forgery). *The attack output is accepted by the strict repaired verifier on the never-queried message M^* .*

Proof. The verifier derives the target vector and address from the serialized public inputs and M^* . Every coordinate opening is valid at that exact address. Lemma 1 makes their ordered combination reconstruct P_A . The copied strict-valid suffix authenticates this unchanged P_A to PK.root . Lemma 2 permits the chosen public inputs. All checks pass, and M^* is fresh by construction. $\square$

Cross-address mixing does not satisfy this proof. The full tree and keypair fields remain in the compressed address, while secret generation, leaf hashing, tree hashing, root compression, and the bottom WOTS key are address separated. The experiment includes a wrong-complete-address rejection control.

4 Exact coverage and concentration

4.1 Coverage at one address

Suppose an address receives r ordinary signatures. For one coordinate, the expected fraction of covered leaves is

$$g_r = 1 - (1 - 1/B)^r. \quad (9)$$

Let Z_A be the fraction of all B^k coordinate vectors covered at address A . Conditional independence of the k disjoint digest fields gives

$$\mathbb{E}[Z_A \mid r] = g_r^k. \quad (10)$$

For the second moment, select two target leaves independently in one coordinate. With probability $1/B$ they agree and have joint coverage g_r ; otherwise both distinct leaves have appeared with probability $1 - 2(1 - 1/B)^r + (1 - 2/B)^r$. Hence

$$s_r = \frac{g_r}{B} + \left(1 - \frac{1}{B}\right) [1 - 2(1 - 1/B)^r + (1 - 2/B)^r], \quad (11)$$

$$\mathbb{E}[Z_A^2 \mid r] = s_r^k. \quad (12)$$

These are exact occupancy identities rather than sparse-Poisson approximations.

4.2 Exact Poisson splitting

Drawing $N \sim \text{Pois}(2^q)$ signatures and assigning each independently to one of 2^h addresses makes all address loads independent $\text{Pois}(\lambda)$ variables, exactly, where

$$\lambda = 2^{q-h}. \quad (13)$$

Write $\mu = \mathbb{E}[Z_A]$ and $\nu = \mathbb{E}[Z_A^2]$. Direct Poisson summation of Equations (10) and (12) gives both quantities. An independent closed-form check follows from the Poisson probability-generating function. Put

$$\alpha = 1 - B^{-1}, \quad \beta = 1 - 2B^{-1}, \quad A_0 = 2 - B^{-1}, \quad C_0 = 1 - B^{-1}.$$

Then

$$\mu = \sum_{j=0}^k (-1)^j \binom{k}{j} \exp(\lambda(\alpha^j - 1)), \quad (14)$$

$$\nu = \sum_{i+j+\ell=k} \frac{k!}{i!j!\ell!} (-A_0)^i C_0^j \exp(\lambda(\alpha^i \beta^j - 1)). \quad (15)$$

The release computes the direct sum with an explicit omitted-tail bound below 2^{-400} and evaluates the closed form independently at high precision. The four headline moment pairs agree within $3.2 \cdot 10^{-13}$ bits.

4.3 Shard mass and a one-run success bound

Preselect a prefix of b address bits, retaining $M = 2^{b-h}$ addresses. Conditional on the realized table, one raw public target trial succeeds with probability

$$p(\mathcal{T}) = 2^{-a'} 2^{-h} \sum_{A \in \mathcal{T}} Z_A, \quad \mathbb{E}[p] = 2^{-a'-b} \mu. \quad (16)$$

Poisson splitting makes the retained Z_A independent. Since $\text{Var}(Z_A) \leq \nu$,

$$\frac{\text{Var}(\sum_A Z_A)}{\mathbb{E}[\sum_A Z_A]^2} \leq 2^{b-h} \frac{\nu}{\mu^2}. \quad (17)$$

Chebyshev therefore bounds the probability that the table has less than half its mean mass by

$$\delta \leq 4 \cdot 2^{b-h} \frac{\nu}{\mu^2}. \quad (18)$$

On the complementary event, fix

$$T = \left\lceil \frac{2}{\mathbb{E}[p]} \right\rceil. \quad (19)$$

Then $p(\mathcal{T})T \geq 1$, so independent ideal- $\mathbf{H}_{\text{msg}}$ target trials fail with probability at most e^{-1} . Before the acquisition-cap abort, success is at least

$$(1 - \delta)(1 - e^{-1}). \quad (20)$$

Every displayed acquisition mean is below 2^{80} . The attack aborts before request 2^{80} , and the Poisson upper-tail correction is negligible. Even for the largest mean 2^{79} , the standard bound at twice the mean is $(e/4)^{2^{79}}$.

5 Concrete resources

5.1 All-address and soft- 2^{80} -byte profiles

Table 2 gives the four optimized rows. Here $Q = 2^q$ is the mean number of ordinary signing queries, T is the fixed fresh-target trial budget, and $Q + T$ is also the number of attacker $\mathbf{H}_{\text{msg}}$ evaluations when one evaluation is used to parse each acquired transcript. “Hard table” is a fully provisioned payload that reserves every coordinate-leaf record at each retained address. “Complete” stores the observed secret and full path; “compact” stores disclosed leaves, a presence bitmap, and the roots of maximal empty sibling subtrees. All entries are base-two logarithms in the unit shown.

The $b = 6$ and $b = 11$ rows are called soft- 2^{80} -byte profiles only because their deterministic table payloads are below 2^{80} bytes. They do not fit the audit host and are not physically

Table 2: Rigorous optimized resource rows; all have success at least 0.6321205588 before the negligible cap abort.

set	shard	$\log Q$	$\log T$	$\log(Q + T)$	returned B	complete B	compact B	hard B	signer cycles
160f	all	70.928	66.640	71.000	85.202	83.154	81.860	85.272	99.864
160f	$b = 6$	71.220	66.934	71.292	85.494	77.385	75.963	79.272	100.156
160s	all	74.760	71.139	74.873	87.968	86.463	84.993	90.729	107.770
160s	$b = 11$	75.587	71.982	75.701	88.795	76.255	74.513	79.729	108.597

runnable. Counting returned data in bits adds three to the displayed byte exponents: $2^{88.494}$ bits for the 160f shard and $2^{91.795}$ bits for the 160s shard. The largest resource in those rows is honest signing, at $2^{100.156}$ and $2^{108.597}$ measured reference cycles. The sub- 2^{80} statement applies to Q and the attacker public- H_{msg} count $Q + T$, not to traffic, storage, or honest-signer work.

At exactly 2^{64} signing queries, the exact fixed-count model gives mean covered-vector exponents -151.0853 and -145.4940 . Including the forced field, one public target would need about $2^{160.0853}$ or $2^{160.4940}$ trials on average. Thus a deployment enforcing a 2^{64} per-key lifetime falls outside this attack range.

5.2 Rows fitting the audit host’s RAM

The NREC audit host exposed 135,045,894,144 bytes ($2^{36.974659}$ bytes) of RAM. Table 3 increases the prefix length until the fully provisioned table fits. The adjacent row gives a larger safety margin. A full implementation would need additional runtime memory, so the safer rows are operational illustrations rather than claims that acquisition can be performed.

Table 3: Machine-RAM shards. “RAM” is the fully provisioned table fraction of 135,045,894,144 bytes.

set, b	$\log Q$	$\log T$	$\log(Q + T)$	returned B	hard B	RAM	signer cycles	success
160f,49	73.538	69.937	73.652	87.812	36.272	0.615	102.474	0.632079
160f,50	73.611	70.066	73.730	87.885	35.272	0.307	102.547	0.632045
160s,54	79.000	78.593	79.811	92.208	36.729	0.843	112.010	0.631807
160s,55	79.000	79.593	80.327	92.208	35.729	0.422	112.010	0.631494

The minimum 160f and 160s tables occupy about 83.0 and 113.9 GB. Under the unrealistically favorable assumptions of perfect scaling over 64 cores at 2 GHz and one measured cycle per nominal cycle, honest signing alone would take approximately $1.74 \cdot 10^{12}$ and $1.29 \cdot 10^{15}$ years. These figures make the distinction precise: this is a concrete query-complexity and parameter-claim attack, not a full attack that can be run on the audit host. The 160s $b = 55$ safety row also crosses 2^{80} attacker public-hash calls; it is included only to show the RAM tradeoff.

5.3 Storage formulas

For completeness, an address with mean occupancy λ has expected distinct leaves per coordinate

$$D = B(1 - e^{-\lambda/B}). \quad (21)$$

At subtree height j , the expected number of empty subtrees whose sibling is nonempty is

$$\frac{B}{2^j} \left(e^{-\lambda 2^j/B} - e^{-\lambda 2^{j+1}/B} \right). \quad (22)$$

Summing Equation (22) gives the compact authentication frontier. The release verifies this identity exhaustively for a small tree. The complete record occupies $((a + 1)n + 1)$ bytes, and a

copied hypertree suffix occupies

$$S = d(4 + \text{len } n) + hn \quad (23)$$

bytes before small table metadata. These formulas reproduce every table exponent above. Simple sharing of identical upper hypertree suffix chunks changes constants but does not reduce a prefix exponent.

6 Experimental validation

6.1 Scaled strict repaired-verifier forgery

The executable strict repaired-verifier model retains the relevant address-separated FORS, Merkle, WOTS, and message-hash structure at reduced dimensions. It generated 240 ordinary randomized signatures under one independently generated key and strictly verified every one. The attack produced a signature accepted on the literal message **fresh-message-never-queried**. Its three FORS coordinates came from three distinct signing queries, its target vector had never appeared at that address, and the complete forged transcript was absent from the acquired transcripts.

The following negative controls all rejected:

1. changing the message;
2. changing serialized R ;
3. changing a FORS authentication node;
4. violating strict WOTS+C membership;
5. attaching a hypertree suffix from another address;
6. substituting a FORS component from another address; and
7. changing the selected FORS index without changing its opening.

The experiment uses fresh randomized signing and an independently generated ordinary key. It does not set, repeat, or roll back signer randomness.

6.2 Full-parameter native FORS-level splice

A second test compiles one common harness against the untouched submitted reference FORS sources for 160f and 160s. At the exact submitted values $(n, a, k) = (20, 7, 30)$ and $(20, 12, 13)$, it generates three openings at one complete address, selects a previously unseen coordinate mixture, and reconstructs the unchanged address-specific public key. All donor openings self-verify. Secret, authentication-path, selected-index, and complete-address mutations change the root.

These are full-parameter FORS-level conformance tests. They do not perform the required address-collision acquisition, public target search, WOTS verification, or an end-to-end full-parameter forgery. The submitted source is useful here because its native FORS core has the fixed-address relation in Equations (6)–(8); the known top-level tree-index bug is outside the harness and supplies no advantage.

6.3 Independent hostile reconstruction

The hostile review used an independent direct Poisson recurrence and did not import the main resource analyzer. For the four rows in Table 2, it reproduced $\log_2 \mu$, $\log_2 T$, and $\log_2(Q + T)$

within $6 \cdot 10^{-12}$ bits. A second implementation evaluated Equations (14)–(15), checked exact WOTS+C membership probabilities by integer dynamic programming, and exhaustively checked the compact-forest formula. Fresh native rebuilds at 160f and 160s matched the prior outputs.

The review’s verdict was “valid abstract attack, not host scale.” It found no hidden reset, randomness collision, weak-key condition, address transplantation, WOTS shortcut, source-only dependency, or uncounted physical resource. The release preserves that verdict and its attack ledger.

7 Scope, lifetime, and prior art

7.1 What is and is not claimed

The result is an ordinary fresh-message forgery in the ideal- H_{msg} model used by the submission. The acquisition oracle returns independently randomized hedged signatures. The attack needs none of the following:

- reset, rollback, repeated signer state, or repeated R ;
- deterministic signing, a chosen-randomness interface, or a fault;
- related keys, weak keys, or a rare hash/key condition;
- the submitted implementation’s collapsed tree index; or
- Algorithm 6’s omitted strict WOTS+C membership check.

The result also does not say that the full-parameter attack was executed or is currently physically practical. Only the scaled end-to-end forgery and full-parameter FORS-level splice were run.

7.2 Per-key lifetime

An exhaustive text audit covered the 53-page specification, the English and Chinese basic-information forms, all reference and optimized source trees, all 16 implementation READMEs, and the security-claim material. It found no maximum signatures-per-key value, mandatory key rotation, or per-key lifetime rule. Section 3.2 instead introduces q_{sig} as “the number of signing queries overall” and expresses the forgery probability as a function of q_{sig} in Equations (3.1)–(3.2), without stating a numerical upper bound on q_{sig} . The separate NGCC Submission Requirements require each key pair to support signing and verifying at least 2^{64} different messages [5]; this is a minimum support requirement, not a maximum per-key lifetime. The NGCC Evaluation Criteria separately permit the attacker to have access to signatures on no more than 2^{80} chosen messages [4].

The precise conclusion is that the specification states no per-key lifetime cap that excludes the attack points reported here, while their signing-query counts remain below the official NGCC chosen-message evaluation bound. This does not attribute an affirmative unlimited-lifetime policy to the submitters. A deployment explicitly and enforceably retiring a key after 2^{64} signatures would fall outside the attack range established here.

7.3 Prior art and novelty boundary

CEDRUS+C Section 3.2 already describes repeated-instance FORS+C accumulation and gives Equation (1). SPHINCS+C introduced the related counter-selected FORS+C design and its security setting [2]. Abri-Katz integrate over few-time-key occupancy and explicitly multiply by a public hash-query count [3]. We claim neither generic leaf accumulation, public replacement of a serialized counter/randomizer, nor the public-query multiplier as new.

At the 3 October 2026 checkpoint, the public CEDRUS+C page listed only the distinct submitted-code tree-index-collapse attack [6]. This is a dated, best-effort statement about public reports, not proof that no unpublished or unindexed analysis exists.

The narrow contribution is the candidate-specific result: a coordinate-splicing algorithm accepted under the strict repaired verifier, exact all-address first and second moments, a concentration-controlled public-target budget, prefix sharding and concrete q_S/q_H optimization for the submitted category-I parameters, a submission-wide lifetime audit, and resource tables that expose rather than suppress the dominant physical costs.

8 Mitigation

The direct operational mitigation is a per-key signature limit chosen jointly with the public-hash budget. A stated 2^{64} lifetime excludes every attack point reported here. Parameter selection should evaluate the aggregate success over all bottom addresses and all permitted public target trials, rather than only the success of one random target at one instance. The same analysis should include returned data, table memory, and honest signing cost when making a practical-security claim.

Changing only the implementation tree-index bug does not repair the attack under the strict repaired-verifier model; full-width indexing is already assumed. Likewise, adding the omitted WOTS+C membership check is sound but irrelevant to the splice, which preserves one complete valid WOTS/hypertree suffix.

9 Conclusion

Ordinary randomized CEDRUS+C signatures expose reusable FORS+C coordinate openings whenever their complete bottom addresses agree. Accumulating those openings over all addresses, then grinding verifier-controlled public message hash inputs, yields a fresh-message forgery with constant success probability. The strict 160f and 160s interaction exponents are approximately 71.0 and 74.9 for all-address tables, or 71.3 and 75.7 when deterministic table payload is kept below 2^{80} bytes.

These figures establish a parameter/lifetime-policy issue under the NGCC evaluation model, not a machine-practical computation. Returned traffic and honest signer work are much larger; actual-host RAM sharding still leaves trillion-year or quadrillion-year signing estimates under idealized scaling. The attack’s structural core and probability bounds survive independent hostile reconstruction, a scaled strict repaired-verifier forgery, and native full-parameter FORS splice tests. Its generic ingredients remain prior art; the new claim is limited to their rigorous CEDRUS+C consequence.

Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

Artifact availability

The accompanying offline reproducer contains the main and independent probability calculations, exact resource tables, a scaled forgery under the strict repaired verifier, native 160f/160s FORS-level splice, negative controls, hostile-review record, frozen expected outputs, and a one-command replay. It verifies its immutable inputs before running, rebuilds native executables from bundled

submitted source, writes generated files only below disposable directories, and ends with an unambiguous `validation=PASS` line.

References

- [1] Siwei Sun, Zhen Qin, Fangyu Zheng, and Zhiyu Zhang, *CEDRUS+C: A Stateless Hash-based Signature Scheme*, NGCC submission specification, 29 June 2026.
- [2] Mikhail Kudinov, Andreas Hülsing, Eyal Ronen, and Eylon Yogev, *SPHINCS+ C: Compressing SPHINCS+ With (Almost) No Cost*, IACR Cryptology ePrint Archive, Report 2022/778, 2022, <https://eprint.iacr.org/2022/778>.
- [3] Mehdi Abri and Jonathan Katz, *Shorter Hash-Based Signatures Using Forced Pruning*, IACR Cryptology ePrint Archive, Report 2025/2069, revised 17 June 2026; CRYPTO 2026, <https://eprint.iacr.org/2025/2069>.
- [4] National Cryptography Administration of China, *Evaluation Criteria for Candidate Algorithms of the Next-Generation Public-Key Cryptography Competition*, Section 1(2), 2025, <https://www.niccs.org.cn/niccs/Notice/1975896137741635584/tT7TSQiz.pdf>.
- [5] National Cryptography Administration of China, *Submission Requirements for the Next-Generation Public-Key Cryptography Competition*, Section 2(2), 2025, <https://www.niccs.org.cn/niccs/Notice/1Dop1mav.pdf>.
- [6] NGCC Security Reports, *CEDRUS+C (sign-03)*, <https://ngcc.dev/reports/sign-03.html>, accessed 3 October 2026.